

Power Measurement Rig — Build Brief

1. The problem

I run software workloads on a Jetson Orin NX 8GB. Each workload is a discrete operation lasting roughly **10–20 seconds**, and a session runs thousands of them back to back over several days.

I need to know **how much energy each individual operation consumed** — not the average over a session, not the board's own estimate, but a real measurement of the actual electrical energy, attributable to one operation and separable from the one before and after it.

The Jetson reports its own power internally, but that reading is the thing I am trying to check, so it cannot also be the reference. I need an independent, externally calibrated measurement to compare it against. That is what this rig is.

Context, only as much as is relevant: this is for a research paper on energy measurement methodology. The credibility of the entire result rests on this rig, so the measurement requirements are unusually strict for something that looks like a simple current-sense job. Where a normal design would accept "close enough," this one cannot.

What "accurate enough" means here

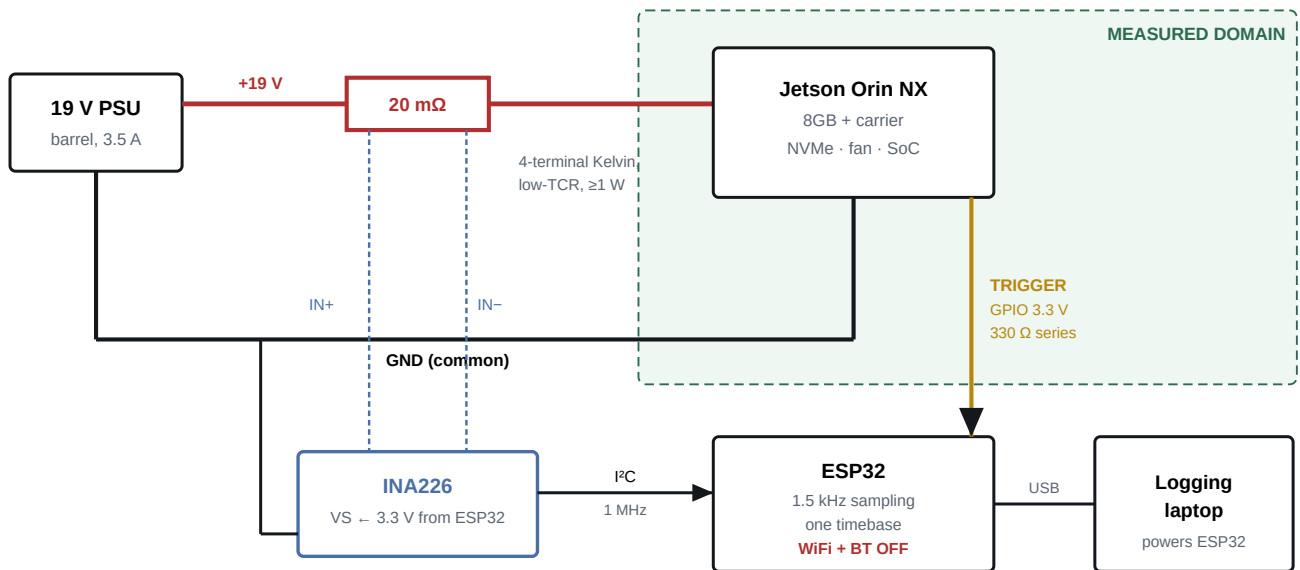
Requirement	Target
Sampling rate	≥ 1 kHz sustained, no dropped samples
Accuracy after calibration	$\leq 1\%$ of reading
Current range	0 – 3.5 A (expected operating point 1.3–1.6 A)
Timing attribution	Operation boundaries resolved to < 1 ms
Stability	Drift bounded and measurable across a 6-day continuous run

2. The solution

An inline high-side current shunt on the Jetson's 19 V DC input, read by an **INA226** current-sense amplifier, sampled by an **ESP32** that also watches a GPIO line from the Jetson marking when each operation starts and stops.

Because the shunt sits on the board's DC input, the measurement covers the whole device — SoC, memory, storage, and fan together. That is intentional.

The Jetson raises a GPIO pin when an operation begins and drops it when the operation ends. The ESP32 timestamps both that pin and the current samples off *the same internal clock*, so the energy and the operation boundary cannot drift apart. Energy for one operation is then the integral of power between the two edges.



Shaded region = everything the shunt sees. The INA226, ESP32 and laptop sit outside it and contribute nothing to the measurement. Dashed sense leads carry no current — that is the Kelvin connection.

The ESP32 is powered by the laptop, NOT by a Jetson USB port.

Complete rig. Only three connections touch the Jetson: the inline power lead, the trigger wire, and ground.

3. Design values

Parameter	Value	Where it comes from
Shunt resistance	20 mΩ	81.92 mV (INA226 full scale) ÷ 4.096 A
Full-scale current	4.096 A	Covers the carrier's 3.5 A rating with margin
Current_LSB	125 μA	$4.096 \text{ A} \div 2^{15}$
INA226 CAL register	2048	$0.00512 \div (125 \text{ μA} \times 20 \text{ mΩ})$
Conversion times	332 μs shunt + 332 μs bus	Gives 664 μs cycle → 1506 Hz
Averaging	1	Averaging done later in software, so nothing is discarded early
I ² C speed	1 MHz	400 kHz leaves too little margin in a 664 μs budget
I ² C pull-ups	1 kΩ – 2.2 kΩ	Rise time limits at 1 MHz — <i>not</i> the usual 4.7 kΩ
Shunt dissipation	45 mW at 1.5 A	245 mW at 3.5 A → use a ≥1 W part, 2 W preferred
Shunt voltage drop	30 mV at 1.5 A	0.16% of 19 V — does not disturb the supply

4. The seven things that must not be changed

These are the constraints where an ordinary, sensible engineering simplification would quietly invalidate the measurement. If any of them looks wrong or impractical, please raise it with me rather than working around it — each one is load-bearing for a reason that isn't obvious from the circuit.

❶ High-side sensing. The shunt goes in the +19 V line, never in the ground return.

Low-side sensing is easier, but it lifts the Jetson's ground above supply ground by $I \times R$ — about 30 mV at 1.5 A, *varying with load*. There is a 3.3 V logic line running between the Jetson and the ESP32, and a ground reference that moves with load moves that line's switching threshold in step with the current being measured. It would couple the measurement into the timing signal. High-side is mandatory here, not a preference.

❷ The ESP32 is powered from the laptop, never from a Jetson USB port.

USB VBUS on the Jetson is downstream of the shunt. If the ESP32 draws from it, the measuring instrument's own consumption gets counted as part of what is being measured. The INA226's 3.3 V supply likewise comes from the ESP32 side, so it too stays outside the measured domain.

3 20 mΩ shunt — check what is actually fitted to the breakout.

Most INA226 modules (CJMCU-226 and clones) ship with a **100 mΩ** shunt. That is a 0.82 A full-scale range; the Jetson would saturate it at roughly half load and the register would simply pin at maximum. The data would look completely plausible and be completely wrong. Either replace the resistor or source a board intended for an external shunt.

4 Kelvin (4-wire) sensing at the shunt.

At 20 mΩ, a couple of milliohms of trace, solder fillet or crimp is a 10% gain error — invisible, because the reading still looks reasonable. Use a 4-terminal current-sense resistor. If only a 2-terminal part is available, solder the sense wires *directly to the resistor's end caps*, inboard of the current-carrying joints. Sense wires carry no current: use thin twisted pair and keep the INA226 close to the shunt.

5 Low temperature coefficient shunt (≤ 50 ppm/°C).

The rig runs continuously for six days and may be calibrated a week before the data is taken. A general-purpose resistor drifts enough over that span to matter. Vishay WSL/WSLP, Bourns CSS, or Susumu KRL families are all suitable.

6 ESP32 firmware: WiFi and Bluetooth never initialised.

The radio stacks take high-priority interrupts that stall the CPU for hundreds of microseconds to milliseconds. The sample period is 664 μs, so those stalls cause missed samples — silently. Data goes out over the wire, not over the air. Related firmware rules: all ISRs marked `IRAM_ATTR`, no flash writes while sampling, serial at 921600 baud or faster (12 kB/s sustained is needed, so 115200 will overflow).

7 Sample on the INA226's conversion-ready interrupt, not a software timer.

Configure the ALERT pin as Conversion Ready and trigger the read from it. A free-running software timer drifts against the sensor's internal conversion clock and will intermittently read the same sample twice or skip one. Either error corrupts an energy total and leaves no trace in the data.

5. Acceptance tests

The rig is done when all of these pass. Each is objective and takes minutes.

Test	Method	Pass criterion
Zero offset	Load disconnected, record 60 s	Mean < 5 mA equivalent; record the value and the standard deviation
Known load	100 Ω 10 W resistor across 19 V	Reads 3.61 W / 190 mA within 2%
Calibration sweep	6 points from 0.2 to 2.5 A against a calibrated bench DMM in series	Linear fit $R^2 > 0.999$, max residual < 1%
Sample integrity	Run 10 minutes under varying load	Sustained ≥ 1.4 kHz with zero gaps in the sequence numbers
Trigger timing	Jetson emits 10,000 pulses of known width; ESP32 measures them	Report median and 99th-percentile error; no missed edges
Shunt thermal	Hold 2 A for 10 minutes	Shunt body rise < 10 °C above ambient
Soak	Run 24 hours continuously	No resets, no dropped samples, no measurable drift in a fixed reference load

6. Bill of materials

Item	Specification	Notes
Current sense amplifier	INA226	Breakout with provision for an <i>external</i> shunt, or bare IC
Shunt resistor	20 m Ω , $\pm 0.5\%$, ≤ 50 ppm/°C, ≥ 1 W, 4-terminal	The critical part. See constraints 3, 4, 5.
Microcontroller	ESP32, ESP32-S3 preferred	S3 has native USB-CDC; either is fine
I ² C pull-ups	2 $\times$ 1 k Ω –2.2 k Ω	Not 4.7 k Ω
Trigger series resistor	330 Ω	Protects both GPIOs against a wiring error
Barrel jack pigtails	5.5 mm OD $\times$ 2.5 mm ID, male + female	Cut the positive lead to insert the shunt inline
Wire	18 AWG current path; thin twisted pair for sense	Keep the current path short
USB isolator (optional)	ADuM3160-class	Breaks the mains ground loop — or just run the laptop on battery
Reference DMM	Calibrated bench meter, 5½ digit+	Borrowed; needed only for calibration sessions

Electronic load	Or a set of power resistors	To set the calibration current points
-----------------	-----------------------------	---------------------------------------

7. Split of work

Electronics subsystem	Me
Shunt selection, sourcing, and inline harness construction	Jetson-side GPIO trigger code
INA226 wiring, pull-ups, Kelvin sense connection	Experiment orchestration and run scheduling
ESP32 firmware: sampling loop, trigger capture, serial streaming	Host-side capture, parsing, and storage
Calibration sweep and the acceptance tests in §5	Energy integration and analysis
Mechanical: mounting, strain relief, enclosure if any	Reporting the rig's specification and calibration in the paper

What I need back from you, in writing, for the paper's methods section: the exact shunt part number, its measured resistance and stated tolerance, its temperature coefficient, the calibration sweep data with the fitted correction, the residual error, and the measured zero-offset and noise floor. These numbers get published, so they need to be real measurements rather than datasheet values.

Questions, or anything here that looks wrong or impractical — please ask before working around it. Several of these constraints exist for non-obvious reasons and I would rather explain one than discover a workaround after six days of data collection.

References: TI INA226 datasheet (SBOS547) · NVIDIA Jetson Orin Nano Developer Kit Carrier Board Specification SP-11324-001.